

UNIVERSIDAD NACIONAL DE CHIMBORAZO FACULTAD DE INGENIERÍA CARRERA DE CIENCIA DE DATOS

Link al git hub: <https://github.com/Miguel-Heredia-UNACH/ACTIVIDAD-AUTONOMA-4-CULTURA-DIGITAL>

1. Introducción

El informe se basa en la optimización de un archivo mal diseñado, con iteraciones innecesarias, problemas de eficiencia y malas prácticas algorítmicas, como puede ser la complejidad de tiempo exponencial ($O(N^2)$), que verifica todos los divisores posibles por cada número iterado, además de creación de listas en memoria con el método `.append()`, perdiendo tiempo y recursos del equipo usado para ejecutar el código.

2. Código Original y Optimización Aplicada

Las técnicas específicas de optimización implementadas incluyeron:

- **Reducción del rango de iteración:** Se limita la comprobación matemática iterando hasta la raíz cuadrada del límite máximo (`np.sqrt(limite)`), lo que reduce las operaciones redundantes.
- **Vectorización con NumPy:** Se sustituyeron las listas estándar de Python por arreglos booleanos (`np.ones`). Mediante el uso de slicing en NumPy (`es_primo[numero*numero : limite+1 : numero]`), eliminando bucles innecesarios en el código original.
- **Extracción eficiente:** Se implementó la función nativa `np.nonzero()` en combinación con metodologías de comprensión para obtener los índices finales de manera óptima.

Captura del código en el Jupyter Notebook:

```
def primo_mal_optimizado(numero_a_evaluar):
    if numero_a_evaluar == 0 or numero_a_evaluar == 1: return False
    lista_de_divisores = []
    for posible_divisor in range(1, numero_a_evaluar + 1):
        if numero_a_evaluar % posible_divisor == 0:
            lista_de_divisores.append(posible_divisor)

    cantidad_de_divisores = 0
    for elemento in lista_de_divisores: cantidad_de_divisores += 1
    return cantidad_de_divisores == 2

def buscar_primos(limite_superior):
    numeros_primos_encontrados = []
    for numero_actual in range(1, limite_superior + 1):
        if primo_mal_optimizado(numero_actual):
            numeros_primos_encontrados.append(numero_actual)
    return numeros_primos_encontrados

t_inicio = time.time()
primos_original = buscar_primos(limite_global)

tiempo_original = time.time() - t_inicio

print(f"Total de números primos encontrados: {len(primos_original)}")
print(f"Tiempo de ejecución original: {tiempo_original:.6f} segundos.")
```

```
def buscar_primos_optimizado(limite):
    if limite < 2: return []
    es_primo = np.ones(limite + 1, dtype=bool)
    es_primo[0:2] = False

    raiz_cuadrada = int(np.sqrt(limite))
    for numero in range(2, raiz_cuadrada + 1):
        if es_primo[numero]:
            es_primo[numero*numero : limite+1 : numero] = False

    return np.nonzero(es_primo) [0].tolist()

t_inicio = time.time()
primos_optimizados = buscar_primos_optimizado(limite_global)

tiempo_optimizado = time.time() - t_inicio

print(f"Total de números primos encontrados: {len(primos_optimizados)}")
print(f"Tiempo de ejecución optimizado: {tiempo_optimizado:.6f} segundos.")
```

3. Resultados y Medición de Tiempos

Se miden los tiempos de ejecución de ambas funciones utilizando la biblioteca `time` y se analizó el rendimiento interno mediante `cProfile`.

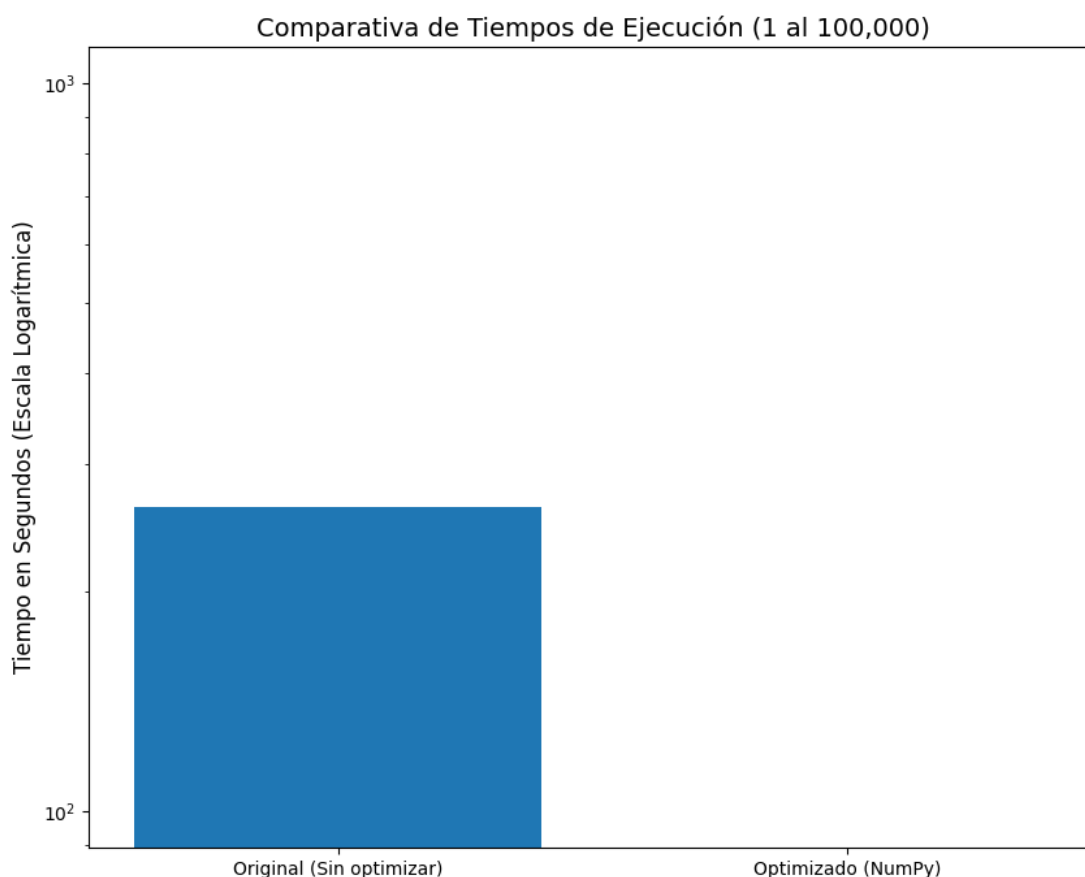
Comparativa de Tiempos de Ejecución:

✓ 4m 21.3s
Total de números primos encontrados: 9592 Tiempo de ejecución original: 261.335513 segundos.
✓ 0.0s
Total de números primos encontrados: 9592 Tiempo de ejecución optimizado: 0.000000 segundos.

Análisis de cProfile: Al utilizar la herramienta de profiling, se generó el archivo `profiling_optimizado.txt`. El análisis revela que en la versión optimizada, el impacto de los bucles de Python es casi inexistente, siendo las llamadas a las funciones internas de la librería NumPy las que representan la mayoría del tiempo total de procesamiento.

1	56 function calls in 0.001 seconds					
2						
3	Ordered by: internal time					
4						
5	ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
6	1	0.000	0.000	0.001	0.001	675330439.py:1(buscar_primos_optimizado)
7	1	0.000	0.000	0.000	0.000	{method 'tolist' of 'numpy.ndarray' objects}
8	1	0.000	0.000	0.000	0.000	{method 'nonzero' of 'numpy.ndarray' objects}
9	2	0.000	0.000	0.000	0.000	{built-in method builtins.compile}
10	1	0.000	0.000	0.001	0.001	4271519845.py:1(<module>)
11	1	0.000	0.000	0.000	0.000	numeric.py:170(ones)
12	2	0.000	0.000	0.000	0.000	codeop.py:120(__call__)
13	1	0.000	0.000	0.000	0.000	{built-in method numpy.empty}
14	2	0.000	0.000	0.001	0.000	interactiveshell.py:3663(run_code)
15	2	0.000	0.000	0.000	0.000	contextlib.py:104(__init__)
16	4	0.000	0.000	0.000	0.000	compilerop.py:180(extra_flags)
17	2	0.000	0.000	0.000	0.000	contextlib.py:141(__exit__)
18	1	0.000	0.000	0.000	0.000	fromnumeric.py:48(wrapfunc)
19	2	0.000	0.000	0.000	0.000	contextlib.py:299(helper)
20	2	0.000	0.000	0.000	0.000	traitlets.py:676(__get__)
21	2	0.000	0.000	0.000	0.000	traitlets.py:629(get)
22	2	0.000	0.000	0.000	0.000	contextlib.py:132(__enter__)
23	5	0.000	0.000	0.000	0.000	{built-in method builtins.getattr}
24	2	0.000	0.000	0.001	0.000	{built-in method builtins.exec}
25	4	0.000	0.000	0.000	0.000	{built-in method builtins.next}
26	1	0.000	0.000	0.000	0.000	fromnumeric.py:1993(nonzero)
27	2	0.000	0.000	0.000	0.000	interactiveshell.py:3615(compare)
28	2	0.000	0.000	0.000	0.000	interactiveshell.py:1299(user_global_ns)
29	4	0.000	0.000	0.000	0.000	typing.py:2287(cast)
30	2	0.000	0.000	0.000	0.000	interactiveshell.py:685(user_ns)
31	1	0.000	0.000	0.000	0.000	fromnumeric.py:1989(_nonzero_dispatcher)
32	2	0.000	0.000	0.000	0.000	{method 'get' of 'dict' objects}
33	1	0.000	0.000	0.000	0.000	multiarray.py:1085(copyto)
34	1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

Visualización Gráfica: A continuación, se presenta la gráfica generada con Matplotlib que ilustra la brecha de rendimiento.



4. Conclusiones

Con este informe, queda claro que la refactorización de un código, demuestra lo importante que es a estructura de datos y el diseño algorítmico en el rendimiento de un programa.

La transición de bucles iterativos puros en Python a operaciones vectorizadas mediante NumPy resultan en el cambio tan significativo de un código que duraba minutos a durar milisegundos. Siempre se recomienda utilizar librerías avanzadas y evitar estructuras complejas que no reflejen en algo positivo al código.